



**INSTITUTE OF INFORMATION TECHNOLOGY
JAHANGIRNAGAR UNIVERSITY
SAVAR, DHAKA-1342**

OPERATING SYSTEM LAB

Course Code: ICT-3102

LAB REPORT NO: 07

Lab Title: Process Scheduling Algorithms

Submitted by:

Md. Shaon Khan

Student ID: 1984

52nd Batch | Session: 2022-2023

Supervised by:

Mehrin Anannya

Assistant Professor

Institute of Information Technology

Jahangirnagar University

Submission Date: July 15, 2026

1. Objective

In this lab I worked on process scheduling, which is basically how the OS decides which process gets the CPU next when there are several of them waiting. The main point was to actually implement three scheduling algorithms myself - FCFS, SJF (Non-Preemptive), and SRTF (the preemptive version of SJF) - and see how each one behaves for the same set of processes.

Along with the implementation, I also had to calculate the usual performance metrics (waiting time and turnaround time) both by hand and through the program, so that I could check whether my code was actually giving correct results. Doing it this way also made it easier to compare the three algorithms and see where each one is strong and where it falls short.

2. Introduction

Process scheduling is one of the core jobs of an operating system. At any given moment a computer usually has more processes wanting to run than it has CPU cores available, so the OS has to pick and choose - this is exactly what a scheduler does. If this wasn't managed properly, the system would either freeze up or become extremely unfair to certain processes.

A few terms that come up repeatedly in this topic:

- **Process:** a program that is currently being executed. It is the basic unit of work for the OS.
- **Process Scheduling:** the decision-making process the OS uses to figure out which waiting process gets CPU time next, and for how long.
- **Ready Queue:** the queue that holds every process which is ready to run but is currently waiting for the CPU.
- **CPU Scheduler:** picks a process out of the ready queue and hands the CPU over to it.

Without any scheduling policy at all, one long process could hog the CPU and everything else would just sit there waiting, which is obviously bad for a multi-tasking system. A good scheduler tries to keep CPU utilization high, keep waiting times reasonably low, and still treat every process somewhat fairly - these are more or less the goals I kept in mind while testing my three implementations below.

3. Basic Terminologies

Before jumping into the algorithms themselves, here are the metrics that were used throughout this report to judge each scheduler:

- **Arrival Time (AT):** the time at which a process shows up in the ready queue.
- **Burst Time (BT):** how much CPU time the process needs in total to finish running.
- **Completion Time (CT):** the time at which the process actually finishes and leaves the system.
- **Turnaround Time (TAT):** total time the process spends in the system, from arrival to completion (waiting + running).

$$TAT = CT - AT$$

- **Waiting Time (WT):** time spent by the process just sitting in the ready queue, not being executed.

$$WT = TAT - BT$$

- **Average Waiting Time:** sum of all the individual waiting times, divided by the number of processes. Lower is generally better.

$$\text{Average } WT = (\Sigma WT) / n$$

- **Average Turnaround Time:** same idea, but for turnaround times.

$$\text{Average } TAT = (\Sigma TAT) / n$$

4. FCFS (First Come First Serve)

Definition

FCFS is the most basic non-preemptive scheduling algorithm out there - whichever process asks for the CPU first simply gets it first, in the exact order the processes lined up in the ready queue.

Rule

Processes run purely by arrival order. Once a process gets hold of the CPU, it keeps running until it's completely done - nobody can cut in line.

Advantages

- Very simple to understand, and honestly not too hard to code either, since it's basically just a queue.
- Fair in terms of order - no process gets special treatment just because it's shorter or more important.
- Low overhead for the OS because there's no complicated logic involved in picking the next process.

Disadvantages

- Suffers badly from what's called the Convoy Effect - if a big process arrives first, every short process behind it has to wait way longer than it should.
- Average waiting time tends to be worse compared to SJF/SRTF.
- Not preemptive at all, so it's not really practical for interactive or time-sharing systems.

Numerical Example

Using the same three-process dataset given in the lab manual:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- t = 0: P1 arrives and since the CPU is free, it starts right away. Finishes at t = 5 (0 + 5).
- t = 5: P2 had already arrived at t = 1, so it goes next. Finishes at t = 8 (5 + 3).
- t = 8: P3 (arrived at t = 2) is the only one left, runs last and finishes at t = 16 (8 + 8).

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (0 + 4 + 6) / 3 = 3.33$$

$$\text{Average Turnaround Time} = (5 + 7 + 14) / 3 = 8.67$$

Algorithm

1. Start.
2. Take the number of processes n and read AT, BT for each one.
3. Sort all the processes by arrival time (ascending).
4. Set $\text{current_time} = 0$.
5. For every process in the sorted order: if $\text{current_time} < \text{AT}$, jump current_time forward to AT (this represents the CPU sitting idle).
6. $\text{CT} = \text{current_time} + \text{BT}$
7. $\text{TAT} = \text{CT} - \text{AT}$, and $\text{WT} = \text{TAT} - \text{BT}$
8. Add these to the running totals, print the row, then set $\text{current_time} = \text{CT}$ and move to the next process.
9. Once every process is handled, divide the totals by n to get the two averages and print them.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

processes.sort(key=lambda x: x[1])
current_time = 0
total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

for process in processes:
    pid, at, bt = process

    if current_time < at:
        current_time = at

    ct = current_time + bt
    tat = ct - at
    wt = tat - bt
    total_wt += wt
    total_tat += tat

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")
    current_time = ct

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Output

```
Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8
```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

```
Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

Explanation

The code follows the algorithm pretty closely. Processes get sorted by arrival time first using a lambda, and then `current_time` keeps track of where we are on the timeline as the loop moves through the list. There's a small check for whenever a process hasn't arrived yet - in that case `current_time` just jumps ahead to its AT so the idle gap is accounted for. CT, TAT and WT are all computed one process at a time and added to running totals, and the final averages printed by the program matched exactly what I got when I worked it out by hand above.

5. SJF (Non-Preemptive)

Definition

SJF (Non-Preemptive) hands the CPU to whichever available process has the smallest burst time. Once it starts running though, it's the same as FCFS in the sense that it can't be interrupted - it has to finish before anything else gets a turn.

Rule

Among all processes that have already arrived by the current time, the scheduler always chooses the one with the smallest burst time.

Advantages

- Gives the best possible average waiting time out of all the non-preemptive algorithms, for a fixed set of processes.
- Works well in batch systems where burst times are already known beforehand.
- Turnaround time tends to be lower overall since shorter jobs clear out of the system fast.

Disadvantages

- Long processes can starve if short ones keep showing up one after another.
- Not very realistic for real-time systems, because you rarely know a process's exact burst time in advance.

Numerical Example

Same three processes again:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- $t = 0$: only P1 has arrived so far, so even though it isn't the shortest job overall, it's the only option. Runs and finishes at $t = 5$.
- $t = 5$: now both P2 (BT=3) and P3 (BT=8) are waiting. P2 is shorter, so it goes next and finishes at $t = 8$ ($5 + 3$).
- $t = 8$: P3 is the only one left. Runs to completion, finishing at $t = 16$ ($8 + 8$).

Interesting side note - for this particular dataset the SJF order ends up being identical to FCFS, since P1 happens to be the only process available at $t = 0$ anyway. The metrics table below comes out exactly the same as the FCFS one.

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (0 + 4 + 6) / 3 = 3.33$$

$$\text{Average Turnaround Time} = (5 + 7 + 14) / 3 = 8.67$$

Algorithm

10. Start.
11. Read n , then AT and BT for each process.
12. Create a completed[] boolean array of size n , all False initially.
13. current_time = 0, completed_count = 0, and reset the total WT/TAT counters.
14. While completed_count < n : look through the process list for one that has arrived ($AT \leq \text{current_time}$), isn't completed yet, and has the smallest BT among those candidates.
15. If nothing qualifies, that means the CPU is idle - just move current_time forward by 1 and check again.
16. Otherwise, for the chosen process: $CT = \text{current_time} + BT$, $TAT = CT - AT$, $WT = TAT - BT$.
17. Add WT/TAT to the totals, print the process details, set current_time = CT, mark it completed, and increment completed_count.
18. After the loop finishes, compute and display the two averages.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

completed = [False] * n

current_time = 0
completed_count = 0
total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

while completed_count < n:
    idx = -1
    min_bt = float("inf")

    for i in range(n):
        pid, at, bt = processes[i]
        if at <= current_time and not completed[i] and bt < min_bt:
            min_bt = bt
            idx = i

    if idx == -1:
        current_time += 1
        continue

    pid, at, bt = processes[idx]
    ct = current_time + bt
    tat = ct - at
    wt = tat - bt

    print(f"P{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

    total_wt += wt
    total_tat += tat

    current_time = ct
    completed[idx] = True
    completed_count += 1

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
```

Output

```
Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8
```

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6

```
Average Waiting Time = 3.33
Average Turnaround Time = 8.67
```

Explanation

Instead of sorting everything once at the start like the FCFS version, this program checks the ready pool on every iteration of the loop, which is closer to how SJF is supposed to behave when multiple processes have already arrived. It scans for every process whose AT is less than or equal to `current_time` and isn't finished yet, then just keeps the one with the smallest BT. If nothing has arrived, a `continue` statement pushes `current_time` ahead by one unit until something becomes available. Running it against the manual example above gave identical numbers, so the logic checks out.

6. SJF (Preemptive / SRTF)

Definition

SRTF is basically SJF but preemptive - at any moment the CPU goes to whichever process has the least remaining burst time, even if that means kicking out a process that's already running.

Rule

At every time unit the scheduler re-checks the ready queue. If a process just arrived and its remaining time is less than what's left on the currently running process, the running process gets preempted and the new one takes over.

What is Preemption?

Preemption just means the OS can pause a running process mid-way and give the CPU to something else, without the first process agreeing to it or finishing what it was doing.

Advantages

- Gives the lowest possible average waiting time among these three algorithms.
- Very responsive - a short process barely has to wait once it arrives.
- Throughput stays high because short jobs are cleared out quickly.

Disadvantages

- Long processes can starve badly if short jobs keep flooding in.
- Lots of context switching overhead since the CPU keeps swapping between processes.
- Needs constant tracking of remaining burst time for every process, which adds complexity.

Numerical Example

Same three processes once more:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	3
P3	2	8

Manual Execution

- $t = 0$: only P1 is around ($BT = 5$), so it starts running.
- $t = 1$: P2 arrives with $BT = 3$. P1's remaining time by now is 4 ($5 - 1$). Since $3 < 4$, P1 gets preempted and P2 takes over.
- $t = 2$: P3 arrives with $BT = 8$. Comparing remaining times - $P1 = 4$, $P2 = 2$, $P3 = 8$ - P2 is still the smallest, so it keeps running.
- $t = 4$: P2 finishes completely. Remaining processes are $P1 = 4$ and $P3 = 8$, so P1 resumes.
- $t = 8$: P1 finally finishes. P3 is the only one left, runs straight through and finishes at $t = 16$ ($8 + 8$).

Calculated Metrics

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P1	0	5	8	8	3
P2	1	3	4	3	0
P3	2	8	16	14	6

$$\text{Average Waiting Time} = (3 + 0 + 6) / 3 = 3.00$$

$$\text{Average Turnaround Time} = (8 + 3 + 14) / 3 = 8.33$$

Compared to FCFS and SJF above, the average waiting time here dropped from 3.33 to exactly 3.00, which lines up with the theory that SRTF should give the best (or tied-best) average waiting time of the three.

Algorithm

19. Start.
20. Read n , then AT and BT for each process.
21. Copy BT into a separate `remaining_bt[]` array so the original burst times stay untouched for the final metric calculations.
22. Initialize `ct[]`, `tat[]`, `wt[]` to zero, and set `current_time = 0`, `completed = 0`.
23. While `completed < n`: find the process with $AT \leq \text{current_time}$, `remaining_bt > 0`, and the smallest `remaining_bt` value among those.
24. If none is found, the CPU is idle - just bump `current_time` up by 1 and loop again.
25. Otherwise: decrement its `remaining_bt` by 1, increment `current_time` by 1.
26. If `remaining_bt` hits exactly 0: increment `completed`, then set $CT = \text{current_time}$, $TAT = CT - AT$, $WT = TAT - BT$ for that process.
27. Once the loop ends, print each process's metrics and sum up WT/TAT to get the averages.

Python Code

```
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time (AT) for P{i+1}: "))
    bt = int(input(f"Enter Burst Time (BT) for P{i+1}: "))
    processes.append([i + 1, at, bt])

remaining_bt = [process[2] for process in processes]

ct = [0] * n
tat = [0] * n
wt = [0] * n

current_time = 0
completed = 0

while completed < n:
    idx = -1
    min_bt = float("inf")

    for i in range(n):
        pid, at, bt = processes[i]
        if at <= current_time and remaining_bt[i] > 0 and remaining_bt[i] < min_bt:
            min_bt = remaining_bt[i]
            idx = i

    if idx == -1:
        current_time += 1
        continue
```

```

    remaining_bt[idx] -= 1
    current_time += 1

    if remaining_bt[idx] == 0:
        completed += 1
        pid, at, bt = processes[idx]

        ct[idx] = current_time
        tat[idx] = ct[idx] - at
        wt[idx] = tat[idx] - bt

total_wt = 0
total_tat = 0

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    pid, at, bt = processes[i]

    print(f"P{pid}\t{at}\t{bt}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

    total_wt += wt[i]
    total_tat += tat[i]

avg_wt = total_wt / n
avg_tat = total_tat / n

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")

```

Output

```

Enter number of processes: 3
Enter Arrival Time (AT) for P1: 0
Enter Burst Time (BT) for P1: 5
Enter Arrival Time (AT) for P2: 1
Enter Burst Time (BT) for P2: 3
Enter Arrival Time (AT) for P3: 2
Enter Burst Time (BT) for P3: 8

```

Process	AT	BT	CT	TAT	WT
P1	0	5	8	8	3
P2	1	3	4	3	0
P3	2	8	16	14	6

```

Average Waiting Time = 3.00
Average Turnaround Time = 8.33

```

Explanation

This one runs the simulation one time unit at a time rather than jumping straight to a process's completion, which is really the only way to model preemption properly. `remaining_bt` keeps the live/updated burst times while the original BT values stay untouched for the final table. On each tick, the loop picks whichever arrived process has the smallest `remaining_bt`, runs it for exactly one unit, and checks if it just hit zero - if so, its CT/TAT/WT get recorded right there. It's a bit more expensive computationally than the other two versions since it re-checks the whole ready pool every single unit of time, but the output matched my manual working out exactly.

7. Comparison of Algorithms

After running all three, here's how I'd sum up the differences between them:

Feature	FCFS	SJF (Non-Preemptive)	SRTF (Preemptive SJF)
Preemptive	No	No	Yes
Selection Basis	Arrival Time	Shortest Burst Time	Shortest Remaining Time
Waiting Time	High	Low	Lowest
Complexity	Easy	Medium	High
Starvation	No	Possible	Possible

8. Result

All three algorithms - FCFS, SJF (Non-Preemptive), and SRTF - were implemented and tested successfully in Python. Each program correctly took in the number of processes along with their arrival and burst times, and worked out the execution order on its own. CT, TAT and WT were calculated for every process in all three cases, and every single value matched the numbers I had already worked out manually, so I'm fairly confident the implementations are correct.

9. Conclusion

Doing this lab made the theory behind CPU scheduling click a lot more than just reading about it. Writing FCFS first made the basic idea obvious, but it was really SJF and especially SRTF that showed why preemption matters - the average waiting time kept dropping as the algorithms got smarter (3.33 → 3.33 → 3.00 for this dataset), which is exactly what the theory predicts, even if the improvement looks small for only three processes. Overall this gave me a much better feel for the trade-off between simplicity and efficiency when it comes to scheduling, and for how the OS actually decides who gets the CPU next.